

用 Codex 系统学习 Python

面向数学、理论物理与 AI for Math 的 24 周学习路线

教材选择 · 仓库结构 · 科学计算 · 软件工程 · Codex 协作 · 能力验收

适用对象

数学训练较强、希望补足工程能力，并将 Python 用于数学研究与计算物理的学习者

版本：2026 年 8 月

使用说明

这份路线的核心目标

不是“学完 Python 语法”，而是能够独立把数学定义转化为可读、可测试、可复现的软件，并能判断 Codex 生成或修改的代码是否真正正确。

建议按 24 周执行。开学后的标准投入为每周约 6 小时；假期可以加速，但不建议跳过独立练习、测试和阶段验收。Python 应逐步成为数学与物理学习的工具，而不是与二者竞争的第三条沉重主线。

半年后的目标状态

- 不使用 AI，也能独立写出数百行规模、结构清晰的 Python 程序。
- 能把数学对象组织成函数、类、模块和包，并为其设计边界测试与不变量测试。
- 掌握 NumPy、SciPy、Matplotlib、SymPy 的核心 workflow。
- 熟练使用 Git、pytest、类型标注、Ruff、mypy 与 GitHub Actions。
- 能阅读、运行和小范围修改陌生 Python 仓库。
- 完成至少一个可公开展示的数学或量子力学项目。
- 能够解释 Codex 的修改、发现其错误，并在空白文件中复现核心实现。

全局原则

原则	具体含义
数学正确性优先	先明确对象、定义域、约定和不变量，再写代码。能运行不等于正确。
独立实现优先	第一次实现核心算法时关闭 Codex；完成后再让其审阅、补测试和提出反例。
测试驱动理解	测试不仅防止回归，还迫使你说明边界、异常和数值误差。
小步提交	每次只改变一个清晰目标，保留可审阅的 Git 历史。
Notebook 与库分离	Notebook 用于探索和展示；可复用逻辑进入 src 包并接受测试。
证明与数值证据分离	数值吻合只能作为证据或检查，不能被描述为数学证明。

1. 教材与资料体系

只设置一条主线教材，其余资料按功能使用。避免同时逐页精读多本入门书，从而把时间消耗在重复内容上。

资料	定位	建议用法
Python Crash Course（第3版）	前8周主教材	重点学习语言基础、函数、类、文件、异常、测试和 Git；大型游戏与 Web 项目可选做。
Think Python（第3版）	概念补充	用于理解函数接口、递归、数据结构与对象模型；选择性阅读，不与主教材并行逐页推进。
Python 官方教程	权威参考	遇到语言细节时查定义；阶段结束后回读对应章节。
Scientific Python Lectures	第9周后的科学计算主线	按 NumPy、Matplotlib、SciPy、调试和高级主题分块学习。
NumPy / SciPy / SymPy 官方文档	查 API 与数值约定	优先查官方文档，不依赖零散博客记忆函数签名。
Effective Python（第3版）	第4个月后选读	已有项目经验后，按当前问题选读函数、类、类型、异常、测试和性能条目。

资料的配合方式

1. 主教材给出知识顺序：本周学什么。
2. 概念教材解释思维模型：为什么这样设计。
3. 官方文档解决精确问题：当前 API、行为和边界是什么。
4. 自己的数学／物理项目形成能力：能否从定义出发独立完成。

不建议的做法

不要把看视频、复制示例、完成书中填空题当作主要进度指标。真正的指标是：能否从空白文件完成任务，能否测试，能否解释，能否修改。

2. 环境与仓库基线

推荐使用 Windows + PowerShell + VS Code/Codex + Git + uv。uv 负责 Python 版本、虚拟环境、依赖和锁文件，减少同时学习多套包管理系统造成的干扰。

推荐工具链

组件	用途	最低要求
Git	版本控制、回退、差异审查	理解工作区、暂存区、提交和分支。
uv	Python、环境、依赖、锁文件与命令运行	所有项目命令通过 uv 执行。
Python 3.14	统一语言版本	由 .python-version 与 pyproject.toml 约束。
pytest	单元测试与参数化测试	会测试正常、边界、异常和浮点结果。
Ruff	Lint 与格式化	用于机械检查，不替代代码理解。
mypy	静态类型检查	公共接口逐步补全类型标注。

组件	用途	最低要求
GitHub Actions	持续集成	每次 push / PR 自动执行质量检查。

仓库结构

```
python-learning-lab/
├── AGENTS.md
├── README.md
├── pyproject.toml
├── uv.lock
├── .python-version
├── exercises/
│   ├── 01_basics/ ... 06_testing/
├── src/mathlab/
├── tests/
├── projects/
│   ├── special_functions/
│   ├── angular_momentum/
│   └── schrodinger_solver/
├── notebooks/
├── docs/
│   ├── syllabus.md
│   ├── progress.md
│   ├── error_log.md
│   ├── concepts.md
│   └── assessment.md
└── .github/workflows/ci.yml
```

四个长期维护文件

文件	作用	记录重点
progress.md	每周学习账本	学了什么、独立完成什么、Codex 参与了什么、下周动作。
error_log.md	错误模型修正	症状、最小复现、错误假设、正确规则、修复和防复发测试。
concepts.md	概念解释清单	用自己的语言解释对象、作用域、迭代器、广播、浮点误差等。
assessment.md	阶段验收	空白文件测试、代码答辩、植入 bug、项目评分。

3. 24 周路线总览

阶段	周数	核心内容	阶段产物
0	第 0 周	环境、Git、uv、测试与 CI	可运行、可测试、可构建的学习仓库
1	第 1-4 周	Python 核心语言	实验数据分析器 + 90 分钟独立验收
2	第 5-8 周	包、类、pytest、类型与 CI	从空目录建立规范 Python 包
3	第 9-13 周	NumPy、数值线代、绘图、SciPy、SymPy	特殊函数数值实验包
4	第 14-18 周	完整量子角动量项目	可测试的角动量与测量工具包
5	第 19-22 周	可复现实验、算法、自动微分与 PyTorch	物理／数学导向的小型 ML 实验
6	第 23-24 周	独立结业项目	独立仓库、技术报告与现场修改

标准时间预算

每周单元	时间	目标
教材学习	90 分钟	阅读一节，手动输入示例，预测并修改输出。
独立练习	90 分钟	关闭 Codex，从空文件完成基础任务。
项目开发	120 分钟	实现一个可测试的小功能。
测试与复盘	60 分钟	补测试、重构、更新错误日志和 Git 记录。
Codex 审阅	约 30 分钟	寻找反例、边界、结构问题和数值风险。

三遍学习法

1. 第一遍 - 理解：阅读、手动运行、预测输出、制造错误。Codex 只解释概念和提出问题。
2. 第二遍 - 独立实现：关闭 Codex，从空文件完成练习并独立调试至少 20-30 分钟。
3. 第三遍 - 工程化：让 Codex 评审接口、测试、边界、类型和重复结构，再由你决定并实施修改。

4. 阶段 0-2：语言与工程基础（第 0-8 周）

周	主题	学习内容	主要任务	验收
第 0 周	建立工作系统	命令行、Git 三种状态、uv 项目、VS Code 调试器、Codex 权限与 diff。	初始化仓库；写 README/AGENTS；创建首个测试；配置 Ruff、mypy 和 CI。	所有本地检查通过；能解释每个生成文件的用途。
第 1 周	表达式、变量与函数	数字、字符串、布尔值、函数、参数、返回值、作用域、基础类型标注。	多项式求值、二次方程、复数极坐标、向量模与内积。	不使用 NumPy；每个函数至少 3 个测试。
第 2 周	条件、循环与基本算法	if、for、while、range、搜索、终止条件与循环不变量。	欧几里得算法、素数判断、二分法、牛顿法、组合数。	能够说明状态如何变化、为什么终止、输出保证是什么。
第 3 周	容器与数据结构	列表、切片、可变性、字典、集合、元组、推导式。	词频、稀疏向量、多项式系数、邻接表、矩阵基础。	为每种结构说明选择理由与复杂度直觉。
第 4 周	文件、路径与异常	pathlib、CSV/JSON、输入验证、异常与命令行参数。	实验数据分析器：读取、验证、统计、报告和保存。	90 分钟独立验收：多项式数据处理器。
第 5 周	模块、包与接口	import、包、__init__、公共接口、src 布局、入口点。	将前四周代码整理为 mathlab 包。	能回答哪些函数应公开、哪些细节应隐藏。
第 6 周	类与数据模型	类、实例、dataclass、组合、继承边界、repr 与相等。	Polynomial 或 StateVector 类，并与纯函数设计比较。	类必须保护有意义的状态或不变量。
第 7 周	pytest	断言、参数化、异常测试、fixture、临时目录、浮点近似。	为此前模块系统补测试。	覆盖正常、零、负值、空输入、非法输入和浮点误差。
第 8 周	类型、格式与 CI	类型标注、Sequence/Iterable、Ruff、mypy、GitHub Actions。	每次 push/PR 自动执行测试、lint、format、type check 与 build。	两小时从空目录建立可安装、可测试的 Python 包。

第 4 周独立验收题

限时 90 分钟；禁止使用 Codex

读取多项式系数和一组输入点，计算函数值、寻找符号变化区间，并将结果输出为 JSON。至少拆成四个函数；包含类型标注、异常处理和不少于八个测试；最后解释每个数据结构的选择。

第 8 周独立验收题

限时 2 小时；Codex 只能解释报错

从空目录建立一个 src 布局的小型 Python 包，包含 pyproject.toml、两个模块、pytest、Ruff、mypy、GitHub Actions 和清晰 README。

5. 阶段 3：科学计算（第 9-13 周）

周	主题	核心内容	主要任务
第 9 周	NumPy 数组	ndarray、shape、dtype、axis、切片与视图、广播、布尔索引、随机生成器。	用纯 Python 与 NumPy 分别实现向量运算；解释广播和 shape 错误。
第 10 周	线性代数与误差	矩阵乘法、线性方程组、特征值、范数、条件数、浮点误差。	构造 J_x 、 J_y 、 J_z ，验证 Hermitian 性、对易关系和 J^2 谱。
第 11 周	Matplotlib	figure/axes、曲线、散点、对数坐标、误差图、保存图片。	绘制特殊函数或角动量对象；绘图代码与计算代码分离。
第 12 周	SciPy	linalg、integrate、optimize、sparse、special。	数值积分、ODE、求根、稀疏本征值；与解析解比较。
第 13 周	SymPy	表达式、微积分、化简、方程、矩阵、lambdify。	比较符号与数值方法；使用符号结果生成数值测试。

阶段项目：特殊函数数值实验

选择一个你在数学物理中实际遇到的函数，例如勒让德多项式、球贝塞尔函数或 Airy 函数，比较下列表示：

- 幂级数或定义式；
- 递推关系；
- SciPy 实现；
- 渐近展开；
- 不同参数区间中的绝对误差与相对误差。

提交物必须同时包含：数学背景、实现、误差图、自动测试、异常区域解释、可重复运行命令。

科学计算的强制习惯

问题	要求
浮点比较	使用合理容差、pytest.approx 或 NumPy testing；说明容差尺度。
数组运算	明确 shape、dtype、axis、广播、复制与视图。
线性代数	优先解线性方程，不显式求逆；关注条件数。
随机实验	显式创建 Generator 并记录种子。
物理对象	测试 Hermitian 性、归一化、守恒量、对易关系等不变量。
结果表述	区分符号恒等式、数值证据和严格证明。

6. 阶段 4：量子角动量工具包（第 14-18 周）

这是最适合与你当前 Sakurai 学习衔接的第一个完整项目。它同时训练线性代数、API 设计、数学不变量、浮点测试、文档、包结构和 CI。

```
quantum-angular-momentum/
├── src/qangular/
│   ├── basis.py
│   ├── operators.py
│   ├── tensor.py
│   ├── states.py
│   └── measurement.py
├── tests/
├── notebooks/
├── docs/
└── examples/
```

功能顺序

1. 自旋 j 的标准基底与索引约定。
2. J_z 、 J_+ 、 J_- ，再由此构造 J_x 、 J_y 与 J^2 。
3. 张量积与复合系统。
4. 两个 spin-1/2 粒子的 singlet / triplet 态。
5. 任意方向自旋测量概率。
6. 密度矩阵与约化密度矩阵。
7. 不同测量方向下的关联函数。
8. 数值验证局域操作不改变远端边缘分布的无信号条件。

每项功能的开发顺序

数学定义 → 接口设计 → 手工可算例子 → 测试 → 实现 → 数值验证 → Codex 审阅

项目硬标准

指标	要求
测试	不少于 40 个，包含不变量、退化情况和回归测试。
文档	公共函数有 docstring；README 有可运行示例。
类型	主要公共接口有完整类型标注。
CI	测试、Ruff、mypy 和 build 全部通过。
设计记录	至少记录 3 个取舍，以及一次重构前后比较。
Git 历史	能看出测试、实现与修正的逐步过程。

7. 阶段 5-6: AI for Math 前置与结业 (第 19-24 周)

周	内容	重点产物
第 19 周	实验可复现性: 随机种子、配置、日志、结果目录、元数据、数据拆分。	一个重复运行能得到同样结果的实验模板。
第 20 周	算法与复杂度: 栈、队列、堆、哈希、图搜索、排序、二分、动态规划直觉。	能够判断 Codex 给出的实现是否明显低效。
第 21-22 周	自动微分与 PyTorch: tensor、shape、计算图、gradient、optimizer、训练/验证/测试。	拟合物理函数、近似微分方程解或比较解析导数与自动微分。
第 23-24 周	独立结业项目: 需求、实现、测试、CI、Notebook、技术报告、现场修改。	可公开展示的独立仓库。

推荐结业题目

- 一维定态 Schrödinger 方程求解器;
- 特殊函数数值实验库;
- 有限维量子系统模拟器;
- 数学表达式数据生成与验证工具;
- 数值线性代数实验平台。

结业提交物

部分	要求
独立仓库	可从 README 复现环境和运行结果。
需求说明	明确问题范围、非目标、输入输出和正确性标准。
数学说明	给出定义、约定、算法和误差来源。
源代码与测试	结构清晰, 核心行为有自动测试。
示例 Notebook	展示问题、实验、图像和结论, 不承担唯一实现。
技术报告	3-5 页, 说明设计、结果、局限与下一步。
现场修改	不依赖 Codex 完成一个小需求或修复一个植入 bug。

8. Codex 的正确角色与使用协议

Codex 应承担“导师、测试者、审阅者和受限实现助手”，而不是持续代写。仓库根目录的 AGENTS.md 用于固化这一协议。

角色	Codex 可以做什么	不应做什么
苏格拉底式导师	找出第一个概念错误，给提示、反例或执行轨迹。	第一次就给出完整答案。
练习生成器	依据当前知识生成递进题目、规格、例子与隐藏测试想法。	同时附上答案。
调试教练	区分症状、错误假设、语言规则和防复发测试。	只告诉你改哪一行。
代码审阅者	检查数学正确性、边界、稳定性、接口、测试和类型。	未经请求直接改写所有文件。
受限实现助手	在规格和测试已明确时完成小范围基础设施或机械实现。	扩大范围、改变公开接口或削弱测试。

推荐提示词

Inspect my current attempt, but do not modify any files.
 Identify the first conceptual mistake that prevents the program from working.
 Give one hint and one minimal counterexample. Do not provide the corrected implementation.

Review my uncommitted changes. Prioritize:

1. mathematical correctness;
2. missing edge cases;
3. numerical stability;
4. unclear interfaces;
5. insufficient tests.

Do not make changes. Report findings with file and line references.

Run the failing test and explain:

1. the observed symptom;
2. the incorrect assumption;
3. the relevant Python rule;
4. the regression test that should prevent recurrence.

Do not edit the code.

必须定期禁止使用 Codex 的任务

- 基础语法练习和 30 行以内的小程序；
- 核心算法的第一次实现；
- 测试用例的基本逻辑；
- 解释报错和逐行追踪；
- 从数学公式推导计算步骤；
- 阶段验收、空白文件测试与代码答辩。

警惕虚假进步

能够清楚描述需求是一种工程能力，但它不能替代独立追踪程序运行、定位错误、判断抽象边界、预测边界行为和写出正确小段代码的能力。

9. 学习成果检验体系

层级	判定标准	是否算掌握
A: 能运行	典型输入上得到结果。	否，只是最低要求。
B: 能测试	主动提出正常、边界、非法、退化和数值困难情况。	部分掌握。
C: 能解释	说明数据流、模块责任、算法终止、复杂度与误差。	是。
D: 能修改	加入新需求、保持旧行为、补测试并进行小型重构。	真正掌握。

四种强制检验

1. 空白文件测试：每 4 周一次，新建空目录，关闭 Codex，限时 90-120 分钟重建核心功能。
2. 代码答辩：随机抽取函数，解释输入输出、不变量、失败情况、复杂度、误差与扩展瓶颈。
3. 人工植入错误：让 Codex 植入一个真实边界 bug，但不透露位置；你独立定位并补回归测试。
4. AI 依赖审计：列出独立完成、提示后完成、共同完成和 Codex 生成的代码，并判断能否复现。

项目 100 分评分表

项目	分值	评价重点
功能与数学正确性	30	定义、约定、不变量、正确结果。
测试质量	20	边界、错误、回归、数值困难情况。
接口与代码结构	15	模块职责、抽象边界、可读性。
可复现性	10	环境、种子、命令和结果记录。
文档	10	README、docstring、数学说明。
Git 历史	5	小步、聚焦、可审阅。
错误处理	5	清晰异常与输入验证。
独立解释和修改	5	脱离 Codex 的答辩与现场任务。

建议标准：70 分为基本合格；80 分可以进入下一阶段；90 分以上适合公开展示。

10. Git 与日常开发 workflow

```

写需求
→ 写或商定测试
→ 创建小分支／检查点
→ 独立实现
→ 运行聚焦测试
→ 运行完整质量检查
→ Codex review
→ 自己修正
→ 检查 diff
→ 提交

```

推荐提交信息

```

test: add boundary cases for polynomial evaluation
feat: implement Horner evaluation
fix: handle empty coefficient sequence
docs: explain numerical error experiment
refactor: separate plotting from computation

```

完成任务前的质量检查

```

uv run pytest
uv run pytest --cov=mathlab --cov-report=term-missing
uv run ruff check .
uv run ruff format --check .
uv run mypy src
uv build

```

没有实际运行的命令不得被描述为“通过”。测试失败时应修复实现或合理修正测试，而不是删除测试、扩大容差或压制警告。

每周复盘模板

```

## Week N

### Learned
- ...

### Independent work
- ...

### Codex-assisted work
- ...

### Errors and corrected assumptions
- ...

### Still unclear
- ...

### Next actions
- ...

```

11. 与数学、理论物理学习的时间协调

在大一阶段，建议优先级为：求真书院数学课程 > 当前理论物理学习 > Python 科学计算与工程训练 > 深度学习框架 > Web、数据库和部署。Python 每周约 6 小时即可，除非当前课程或研究项目明确需要加量。

把编程嵌入现有学习

正在学习的内容	对应 Python 任务
角动量	构造矩阵、验证对易关系、实现 Clebsch-Gordan 相关计算。
微分方程	比较 Euler、Runge-Kutta、边值问题与解析解。
特殊函数	绘图、递推、级数、渐近展开与误差区域。
线性代数	条件数、本征值算法、稀疏矩阵和数值稳定性。
概率	模拟、置信区间、随机种子与收敛实验。
量子测量	密度矩阵、测量概率、约化态与关联函数。

时间分配判断

某项 Python 练习如果既不服用于当前数学／物理内容，也不训练路线中尚缺的工程能力，应优先删除，而不是因为“可能以后有用”而保留。

前七天执行清单

天	任务
第 1 天	安装 Git、VS Code/Codex 和 uv；配置 Git；创建本地仓库。
第 2 天	放入 AGENTS.md，让 Codex 初始化结构、环境、README、测试和 CI；逐项审查 diff。
第 3 天	学习变量、表达式和函数；独立实现 norm2、quadratic_roots、evaluate_polynomial。
第 4 天	为上述函数编写 pytest：一般、零、重根、非法输入、浮点比较。
第 5 天	练习 status、diff、add、commit、log、switch、分支与合并。
第 6 天	让 Codex 只做代码审阅，不修改；自己处理审阅意见。
第 7 天	关闭 Codex，从空文件重写多项式求值与二次方程，并完成周复盘。

附录 A：如何使用随附的 AGENTS.md

1. 在一个空文件夹中初始化 Git，或直接用 VS Code / Codex 打开该文件夹。
2. 把随附的 AGENTS.md 放到仓库根目录。
3. 向 Codex 发送：“请按照 AGENTS.md 的 bootstrap protocol 初始化这个学习仓库，完成后运行全部检查并报告结果。”
4. 审查 Codex 的命令、权限请求和 diff。机器级安装、网络、远程仓库和 push 应保持显式批准。
5. 初始化完成后，Codex 默认进入教学模式：它可以审阅和提示，但不会自动代写核心练习。

AGENTS.md 的设计特点

- 一次性 Bootstrap 与长期 Teaching Mode 分离。
- 环境初始化是幂等的：先检查，保留已有工作，不盲目覆盖。
- 规定仓库结构、依赖、CI、质量命令和文档模板。
- 要求 Codex 对数学正确性、数值稳定性和不变量进行审阅。
- 建立提示阶梯，避免第一次就透露完整答案。
- 限制 destructive Git、密钥、push、发布和工作目录外写入。

附录 B：权威资料与更新说明

工具和在线文档会持续更新。下列链接用于核对当前行为；本路线的工具事实已按 2026 年 8 月可查的官方文档整理。

1. OpenAI Codex - Custom instructions with AGENTS.md:
<https://developers.openai.com/codex/agent-configuration/agents-md>
2. OpenAI Codex - Best practices: <https://developers.openai.com/codex/learn/best-practices>
3. OpenAI Codex - Windows sandbox: <https://developers.openai.com/codex/windows/windows-sandbox>
4. OpenAI Codex - Windows desktop app: <https://developers.openai.com/codex/windows/windows-app>
5. uv - Working on projects: <https://docs.astral.sh/uv/guides/projects/>
6. uv - Using uv in GitHub Actions: <https://docs.astral.sh/uv/guides/integration/github/>
7. Python 3.14 official tutorial: <https://docs.python.org/3/tutorial/>
8. Python Crash Course, 3rd Edition - No Starch Press: <https://nostarch.com/node/691>
9. Think Python, 3rd Edition: <https://alldowney.github.io/ThinkPython/>
10. Scientific Python Lectures: <https://lectures.scientific-python.org/>
11. pytest documentation: <https://docs.pytest.org/>
12. mypy documentation: <https://mypy.readthedocs.io/en/stable/>
13. GitHub Docs - Building and testing Python:
<https://docs.github.com/en/actions/tutorials/build-and-test-code/python>
14. NumPy user guide: <https://numpy.org/doc/stable/user/>
15. SciPy documentation: <https://docs.scipy.org/doc/scipy/>
16. SymPy documentation: <https://docs.sympy.org/latest/>

注：GitHub Actions 与第三方 Action 的版本会变化。AGENTS.md 要求 Codex 在生成 CI 时核对当前官方文档，并尽可能使用官方示例中的完整提交 SHA。